

Design and train autoencoder model for image compression

```
import tensorflow as tf
from tensorflow.keras.layers import Input, Conv2D, MaxPooling2D, UpSampling2D
from tensorflow.keras.models import Model
import numpy as np
import matplotlib.pyplot as plt
```

```
(x_train, _), (x_test, _) = tf.keras.datasets.mnist.load_data()
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0
x_train = np.expand_dims(x_train, axis=-1)
x_test = np.expand_dims(x_test, axis=-1)
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>
11490434/11490434 — 0s 0us/step

```
input_img = Input(shape=(28, 28, 1))
```

```
# Encoder
x = Conv2D(32, (3, 3), activation='relu', padding='same')(input_img)
x = MaxPooling2D((2, 2), padding='same')(x)
x = Conv2D(16, (3, 3), activation='relu', padding='same')(x)
x = MaxPooling2D((2, 2), padding='same')(x)
x = Conv2D(8, (3, 3), activation='relu', padding='same')(x)
encoded = MaxPooling2D((2, 2), padding='same')(x)
```

```
# Decoder
x = Conv2D(8, (3, 3), activation='relu', padding='same')(encoded)
x = UpSampling2D((2, 2))(x)
x = Conv2D(16, (3, 3), activation='relu', padding='same')(x)
x = UpSampling2D((2, 2))(x)
x = Conv2D(32, (3, 3), activation='relu')(x)
x = UpSampling2D((2, 2))(x)
decoded = Conv2D(1, (3, 3), activation='sigmoid', padding='same')(x)
```

```
# Compile the autoencoder
autoencoder = Model(input_img, decoded)
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')
```

```
autoencoder.fit(x_train, x_train, epochs=5, batch_size=128, shuffle=True, validation_data=(x_test, x_test))
```

```
decoded_imgs = autoencoder.predict(x_test[:10])
```

Epoch 1/5
469/469 — 113s 233ms/step - loss: 0.2851 - val_loss: 0.1409
Epoch 2/5
469/469 — 136s 221ms/step - loss: 0.1362 - val_loss: 0.1218
Epoch 3/5
469/469 — 143s 223ms/step - loss: 0.1209 - val_loss: 0.1136
Epoch 4/5
469/469 — 140s 219ms/step - loss: 0.1132 - val_loss: 0.1080
Epoch 5/5
469/469 — 101s 216ms/step - loss: 0.1089 - val_loss: 0.1048
1/1 — 0s 304ms/step

```
# Get compressed (encoded) representations
encoder = Model(inputs=input_img, outputs=encoded)
encoded_imgs = encoder.predict(x_test)
```

```
# Print original and compressed shapes for one image
original_shape = x_test[0].shape
compressed_shape = encoded_imgs[0].shape
```

```
print("Original image shape:", original_shape)
print("Compressed image shape:", compressed_shape)
```

```
# Print actual number of elements (bytes, sort of)
original_size = np.prod(original_shape)
compressed_size = np.prod(compressed_shape)
```

```
print("Original image size (pixels):", original_size)
print("Compressed image size (pixels):", compressed_size)
print("Compression ratio: {:.2f}x".format(original_size / compressed_size))
```

313/313 — 3s 10ms/step
Original image shape: (28, 28, 1)
Compressed image shape: (4, 4, 8)
Original image size (pixels): 784

Compressed image size (pixels): 128
 Compression ratio: 6.12x

```
n = 10

plt.figure(figsize=(20, 4))
for i in range(n):
    # Original
    ax = plt.subplot(2, n, i + 1)
    plt.imshow(x_test[i].reshape(28, 28), cmap='gray')
    plt.axis('off')

    # Reconstructed
    ax = plt.subplot(2, n, i + 1 + n)
    plt.imshow(decoded_imgs[i].reshape(28, 28), cmap='gray')
    plt.axis('off')
plt.show()
```



```
encoder = Model(inputs=input_img, outputs=encoded)
encoded_imgs = encoder.predict(x_test)
print("Encoded shape:", encoded_imgs.shape)
```

313/313 ————— 2s 7ms/step
 Encoded shape: (10000, 4, 4, 8)